DESARROLLADOR DE APLICACIONES

Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

0484 MP Bases de Datos
UD1 Almacenamiento de la Información

Índice

01. Ficheros

02. Bases de datos

2.1 De los sistemas de ficheros a los sistemas de bases de datos
2.2 Conceptos de bases de datos
2.3 Niveles de abstracción en una base de datos
2.4 Tipos de bases de datos según el modelo de datos

03. Sistemas gestores de bases de datos

3.1 Funciones de un SGBD
3.2. Componentes de un SGBD
3.3. Tipos de un SGBD
3.4. Bases de datos centralizadas y distribuidas

04. Bibliografía

01. Ficheros

Un ordenador almacena muchos tipos de información: datos administrativos, contables, música, imágenes, películas, páginas webs, etc. Toda esta información está almacenada en los dispositivos de almacenamiento del ordenador, como discos duros, pen drives, DVDs etc., y se organiza mediante ficheros o archivos.

Por lo tanto, los ficheros son estructuras de información que crea el sistema operativo para poder almacenar datos. Tienen un nombre y una extensión, que determina el formato o tipo de la información que contienen.

El formato o tipo de fichero determinan la forma de interpretar la información que contiene, ya que, en definitiva, lo único que se almacena en un fichero es una ristra de ceros y unos, (es decir bits, por ejemplo 00110001111110) de forma que es necesaria su interpretación para dar sentido a la información que se almacena. El sistema operativo o la utilidad que trate cada fichero debe conocer el formato que contiene para poder representar la información, ya que por ejemplo si se intenta abrir un archivo .docx con un bloc de notas, la información que mostrará será ilegible, pues el bloc de notas no sabe interpretar el contenido del fichero.

Tradicionalmente, los ficheros se han clasificado de muchas formas, algunas de estas clasificaciones son:

Según su contenido:

- **Ficheros de texto:** los bits almacenados en el fichero son traducidos por el sistema operativo a caracteres alfanuméricos y números que en tiende el ser humano.
- **Ficheros binarios:** los bits almacenados en el fichero son tratados como componentes de estructuras de datos más complejas, como ficheros que almacenan sonido, vídeo, imágenes, etc.

Según su organización: es decir, según la forma en que se han de acceder a los datos.

- **Ficheros de acceso secuencial:** en este tipo de ficheros los datos están dispuestos siguiendo una secuencia ordenada, es decir, unos detrás de otros. Se caracterizan por tener que recorrer todos los datos anteriores para llegar a uno en concreto.

- **Ficheros de acceso directo:** permiten acceder a un dato en concreto sin necesidad de acceder a todos los anteriores.
- **Ficheros de acceso indexado:** se accede a los datos consultando un índice, es decir, una estructura de datos que permite acceder a la información rápidamente, simulando la forma en que el índice de un libro facilita el acceso a sus contenidos.

Generalmente los ficheros que componen una base de datos son de tipo binario, puesto que la información que hay almacenada en ellos debe tener una estructura lógica y organizada para que las aplicaciones puedan acceder a ella de manera universal, es decir, siguiendo un estándar. Esta estructura lógica y organizada, generalmente es muy difícil de expresar mediante ficheros de texto, por tanto, la información de una base de datos se suele guardar en uno o varios ficheros binarios. Por ejemplo, Access guarda toda la información de la base de datos en un fichero binario con extensión .mdb.

02. Bases de datos

2.1. De los sistemas de ficheros a los sistemas de bases de datos

Inicialmente, las primeras empresas que empezaron a usar sistemas informáticos trabajaban con sistemas de ficheros. Es decir, trabajaban con programas que manejaban información almacenada en ficheros, donde cada equipo trabajaba con sus propios datos y programas, y se encargaba de su mantenimiento y gestión.

Con el incremento de la cantidad de información, así como de los usuarios que la manejaban empezaron a surgir los problemas de integridad, duplicidad de información, seguridad, etc.

Estos problemas llevaron a la organización de la información mediante un sistema más ordenado y manejable basado en la centralización de la gestión y la organización de los datos en forma de bases de datos.

Cabe destacar que en última instancia todo se almacena en ficheros, pero cuando usamos bases de datos no trabajamos directamente con ficheros, sino con estructuras de datos más fáciles de manejar.

Veamos estos problemas con un ejemplo. Supongamos una empresa que tiene los siguientes departamentos:

- Un departamento de **pedidos** donde se gestiona todo lo relativo al pedido de un cliente.
- Un departamento de **marketing** donde se trabaja con los clientes para ofrecerles descuentos, publicidad, ofertas, etc.

Supongamos que tenemos un fichero de clientes que se maneja en ambos departamentos. Rellenamos una solicitud de un pedido de un cliente con los siguientes datos:

Nº pedido: 56 **Cliente**: Juan García López **Dirección**: C\ Pez, 6 **Teléfono**: 619 123 456	
ARTÍCULO	**UNIDADES**
005	3
018	2

Con este diseño estamos repitiendo información, lo que se denomina **redundancia de datos**:

- Los datos del cliente estarán repetidos en cada pedido que realice.
- Además, estarán en dos departamentos distintos (marketing y pedido).

Como se puede deducir fácilmente, se está gastando espacio de almacenamiento sin necesidad, pero además ¿qué ocurre si cambia el teléfono de un cliente?

- Deberá cambiarse en todos los pedidos de ese cliente en el departamento de pedidos.
- Deberá cambiarse en los datos del cliente que estén en el departamento de marketing.
- Si no se hace así puede haber dos campos teléfono con distintos valores.

Esto se denomina **inconsistencia de datos**: la posibilidad de valores distintos para un mismo dato.

La solución a estos problemas (y muchos otros que existen si se guarda la información en ficheros) consiste en **integrar los datos**, guardando los datos una sola vez y permitiendo que se compartan. Así es como surgen los **sistemas de bases de datos**, que podemos definir como el conjunto de datos relacionados entre sí, con las mínimas repeticiones posibles, que pueden ser procesados simultáneamente por varios programas o usuarios y los **sistemas de gestión de**

bases de datos (SGBD) que sería el software que me permite crear y manipular los datos de la base de datos.

2.2. Conceptos de bases de datos

Como hemos comentado anteriormente, las **bases de datos** surgen como alternativa a los sistemas de ficheros, intentando eliminar o al menos reducir sus inconvenientes, y con ellas surgen también los **sistemas gestores de bases de datos (SGBD)** como herramienta que permite la implementación y gestión de bases de datos.

Una base de datos es un conjunto de datos almacenados relacionados entre sí que satisfacen las necesidades de información de una empresa u organización.

Algunas características importantes de las bases de datos que interesa resaltar son:

- La base de datos es un conjunto de datos organizados en estructuras que se definen una sola vez y que se utilizan al mismo tiempo por muchos equipos y usuarios.
- En lugar de almacenarse en ficheros desconectados y de manera redundante, los datos en una base de datos están centralizados y organizados, de forma que se minimice la redundancia y se facilite su gestión.
- La base de datos es única para toda la organización, es decir no pertenece a un solo equipo.
- La base de datos debe estar formada no solamente por los datos de la organización, sino también por una descripción de dichos datos (lo que llamamos los metadatos), que se almacena en el diccionario de datos o catálogo.

Las bases de datos no sólo presentan ventajas, sino que también tienen posibles inconvenientes que son necesarios valorar antes de tomar una decisión relativa a un cambio en la manera de almacenar los datos. Entre estos inconvenientes es preciso destacar:

- **Instalación costosa**: la implantación de un sistema de bases de datos puede llevar consigo un coste elevado, tanto en equipo físico (nuevas instalaciones o ampliaciones), como en el lógico (sistemas operativos, programas, compiladores, etc. necesarios para su uso).
- **Personal especializado**: los conocimientos, que resultan imprescindibles para una utilización correcta y eficaz y sobre todo para la administración de las bases de datos, implican una necesidad de personal especializado.

- **Implantación larga y difícil**: la implantación de una base de datos puede convertirse en una tarea larga y laboriosa.
- **Falta de rentabilidad a corto plazo**: la implantación de un sistema de bases de datos, tanto por su coste en personal y en equipos como por el tiempo que tarda en estar operativo, no resulta rentable a corto plazo.
- **Ausencia real de normas**: un problema muy importante que se pone de manifiesto en el momento de la creación de una base de datos es la ausencia real de una estandarización que facilite a los usuarios el manejo de los sistemas de bases de datos. Empieza, sin embargo, a observarse ya una preocupación por este tema y van apareciendo algunos estándares, sobre todo en el campo de las bases de datos relacionales como el SQL.

2.3. Niveles de abstracción en una base de datos

El comité ANSI SPARC (*American National Standard Institute Standard Planning and Requirements Comittee*) propuso en 1975 un estándar para la creación de sistemas de bases de datos basado en una arquitectura de tres niveles, que ha marcado el desarrollo de los sistemas gestores de bases de datos (SGBD).

El objetivo de la arquitectura de tres niveles es el de separar en niveles de abstracción el esquema de una base de datos. Son tres formas distintas de ver o representar una misma base de datos. Es decir, cada nivel describe los mismos datos, pero con distinto nivel de abstracción, fijándose en distintos detalles. Los únicos datos que existen están realmente almacenados en un dispositivo físico, por ejemplo, en un disco duro.

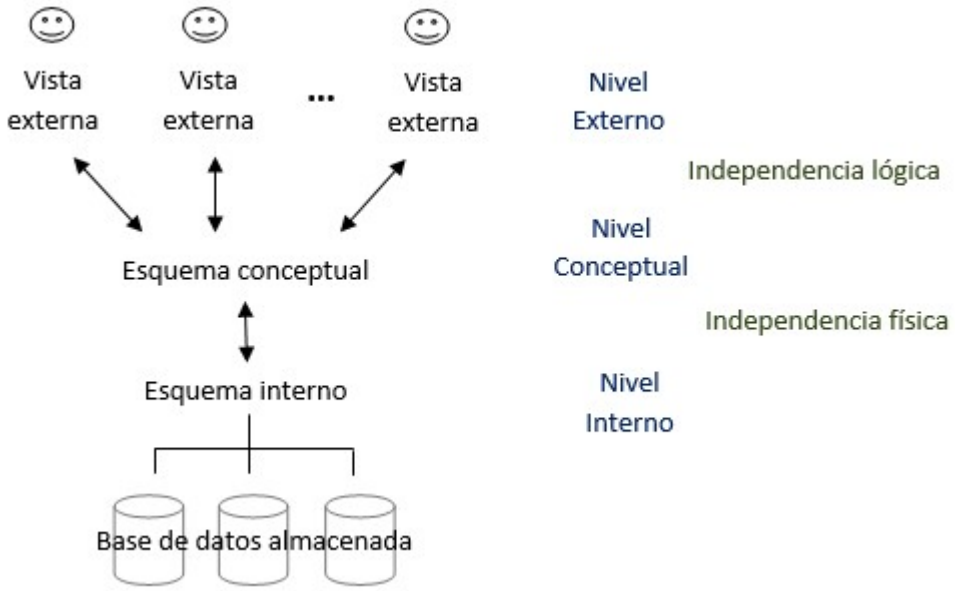
Los tres niveles de la arquitectura son:

- **Nivel interno**: describe la estructura física de almacenamiento de la base de datos, es decir, cómo están almacenados los datos (unidades de disco, archivos y carpetas del sistema). Este nivel afecta al administrador del sistema.
- **Nivel global o conceptual**: se describe el esquema teórico de cómo se organizan los datos y se interrelacionan, es decir se lleva a cabo el diseño de la base de datos. Es independiente de la máquina y de la visión de cada usuario particular.
- **Nivel externo**: Se describe cómo se perciben los datos por el usuario final. Es la visión de los datos que poseen los usuarios finales a través de las aplicaciones que manejan la base de datos. Habrá tantos esquemas como visiones diferentes haya.

Esta arquitectura de tres niveles nos va a marcar la independencia de datos, concepto muy importante en bases de datos. La independencia de datos es la capacidad de alterar el esquema que define un nivel, sin necesidad de alterar su nivel superior. Como muestra la siguiente imagen los tipos de independencia de datos son:

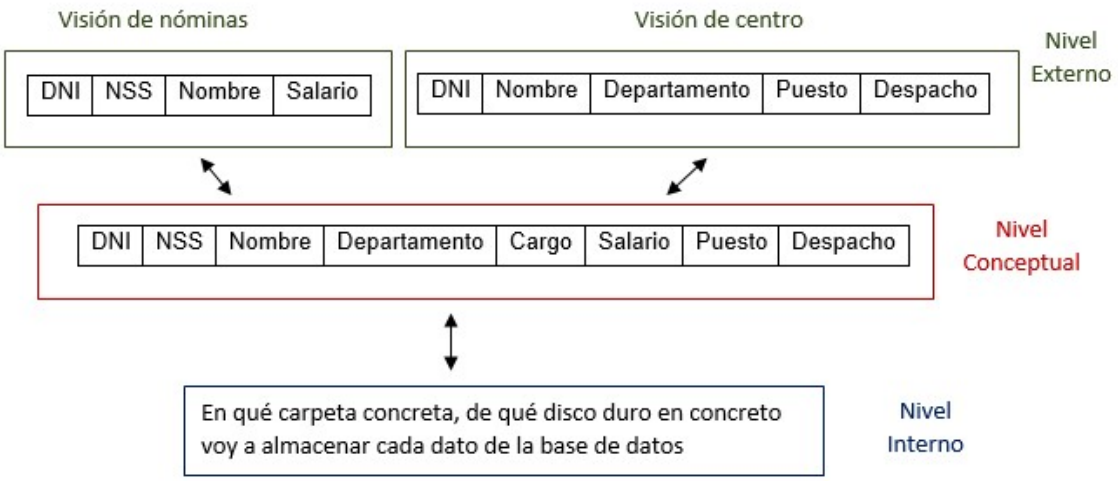
- **Independencia lógica**: Es la capacidad de modificar los esquemas del nivel conceptual sin tener que modificar las vistas de los usuarios, es decir el esquema externo. El esquema conceptual puede verse modificado si necesitamos ampliar o reducir la base de datos.
  Por ejemplo, si en la base de datos de una universidad no deseamos almacenar de ahora en adelante información sobre el estado civil de los alumnos, se eliminaría este atributo en el esquema conceptual, pero los esquemas externos que no se refieran al estado civil del alumno no se ven afectados.

- **Independencia física**: Es la capacidad de modificar el esquema interno sin tener que alterar los esquemas de los niveles conceptuales o externos.
  Por ejemplo, si se añaden o cambian discos u otro hardware, o se modifica el sistema operativo u otros cambios relacionados con la física de la base de datos, el esquema conceptual permanece invariable.



El objetivo de la arquitectura en tres niveles es conseguir que el esquema conceptual sea una descripción estable de los datos de la organización e independiente la

manera que dichos datos son almacenados físicamente en disco y de la manera que se muestran al usuario (vistas).



2.4. Tipos de bases de datos según el modelo de datos

La base de datos consiste en los datos concretos referentes a un sistema o parte del mundo que hemos modelado, por ejemplo, la base de datos de una universidad incluye todos los datos relativos a alumnos, profesores, notas, etc. Estos datos son sencillos de manejar cuando son pocos, pero cuando su volumen crece se requiere el uso de distintos modelos para facilitar el diseño de la base de datos.

Un **modelo de datos** es conjunto de herramientas conceptuales útiles para describir los datos, las relaciones entre ellos, la semántica asociada a los datos y las operaciones que pueden hacerse sobre ellos. Un modelo de datos nos proporciona los mecanismos de abstracción para representar una parte del mundo cuyos datos nos interesan.

En un principio existían tres tipos de bases de datos según su modelo de datos bases de datos jerárquicas, en red y relacionales. Más recientes son las bases de datos orientadas a objetos. A continuación, explicaremos brevemente cada una de ellas.

Bases de datos jerárquicas:

Las bases de datos jerárquicas organizan la información en forma de árbol invertido, en las que existen nodos padres que pueden tener unos varios nodos hijo.

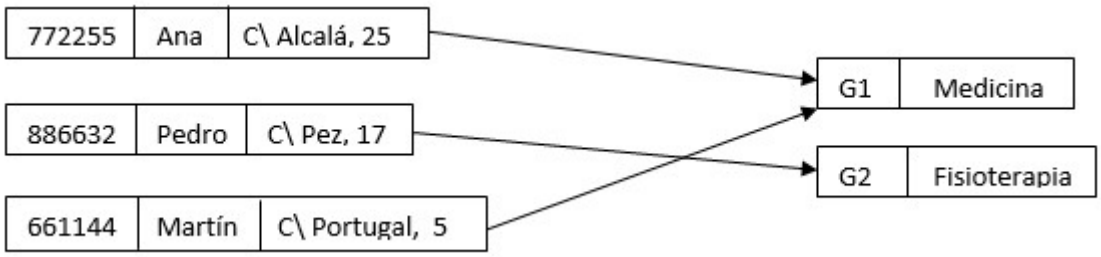
Para sistemas que manejan una gran cantidad de información y, donde esos datos además están muy compartidos, son útiles este tipo de bases de datos jerárquicas puesto que permiten crear estructuras muy estables. Sin embargo, la incapacidad de almacenar de manera eficiente la duplicidad de datos es su gran desventaja. Actualmente está base de datos está en desuso.

Bases de datos en red:

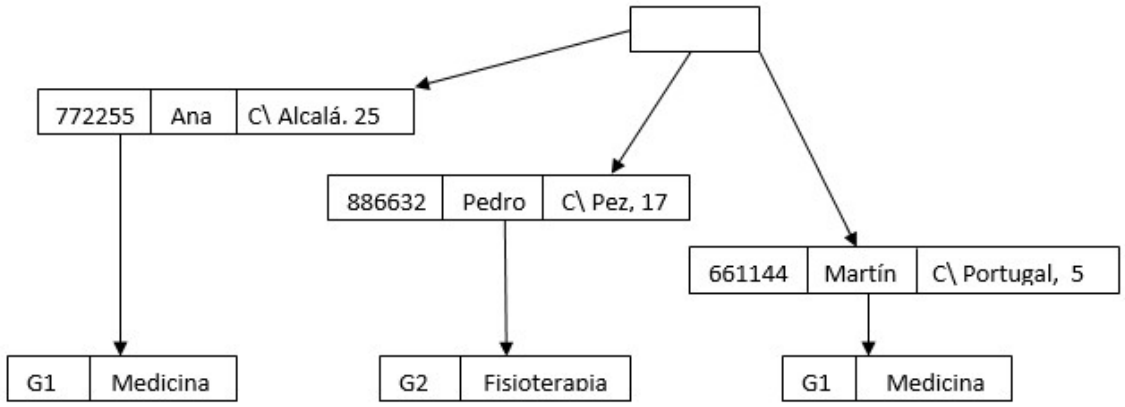
Este tipo de bases de datos se crea para mejorar las limitaciones de las bases de datos jerárquicas vistas anteriormente. Al igual que en el modelo jerárquico, la información se almacena en nodos padres e hijo, a diferencia de él, este modelo en red sí se permite que un nodo hijo tenga varios nodos padres.

Esta mejora ofrece una manera de solventar el problema de la duplicidad de la información, pero la dificultad para administrar la información en una base de datos de red ha hecho que este modelo haya caído en desuso.

Por ejemplo, la representación de lo que podría ser parte de la base de datos de una universidad en la que se muestran los estudios cursados por cada alumno sería:



Esta misma información representada en un modelo jerárquico tendría, como muestra la imagen siguiente muchas más redundancias de información.



La primera generación de los SGBD la constituyen los sistemas jerárquicos y en red. Sin embargo, los inconvenientes de estos sistemas (como por ejemplo la inexistente independencia de datos o controles de seguridad) hicieron que este tipo de bases de datos se sustituyan por los sistemas relacionales que explicamos a continuación.

Bases de datos relacionales:

Las bases de datos relacionales representan los datos y las relaciones entre estos a través de una colección de tablas en las cuales:

- Las **filas (tuplas)** son cada uno de los registros que contendrá la base de datos.
- Las columnas son **características (atributos)** de cada registro localizado en la tupla.

Éste es el modelo utilizado en la actualidad para el almacenamiento y administración de los datos, y por tanto es el modelo que se estudiará fundamentalmente a lo largo del curso.

Usando bases de datos relacionales, el ejemplo visto anteriormente en las bases de datos jerárquicas y en red sobre los estudios cursados por cada alumno, quedaría de la siguiente manera:

ESTUDIANTES			
DNI	**Nombre**	**Dirección**	**Código_grado**
772255	Ana	C\ Alcalá, 25	G1
886632	Pedro	C\ Pez, 17	G2
661144	Martín	C\ Portugal, 5	G1

GRADOS	
Código_grado	**Grado**
G1	Medicina
G2	Fisioterapia

Bases de datos orientadas a objetos:

Es un modelo más reciente en el que la base de datos está formada por **objetos** relacionados entre sí, siendo los objetos entidades con un **estado** o datos asociados, y un **comportamiento** o funcionalidad determinada.

Los conceptos del paradigma de la orientación a objetos se incorporan a este tipo de bases de datos. Entre ellos destacan:

- Encapsulación: garantiza los accesos incorrectos a los datos, gracias a la ocultación de información al resto de objetos.
- Herencia: permite crear objetos a partir de objetos existentes de los que heredan atributos y métodos (comportamiento).
- Polimorfismo: métodos que pueden aplicarse a distintos tipos de objetos.

03. Sistemas gestores de bases de datos

El sistema gestor de la base de datos (SGBD) es una aplicación que permite a los usuarios definir, crear y mantener la base de datos y proporciona acceso controlado a la misma. Es una herramienta que sirve de interfaz entre el usuario y la base de datos.

Un SGBD es un conjunto integrado de programas que proporciona los medios para describir, recuperar y manipular datos, manteniendo su integridad y confidencialidad.

3.1. Funciones de un SGBD

Las funciones que deben cumplir los SGBD son:

- **Permiten almacenar datos, acceder a ellos y actualizarlos** de forma sencilla y ocultando las características físicas de los dispositivos donde son almacenados.
- **Garantizan la integridad y consistencia de los datos**, ya que los datos duplicados o derivados deben de mantener sus valores coherentemente. Por ejemplo, si tengo un campo año de nacimiento, el campo edad es un campo derivado de él (ya que la edad se puede obtener a partir de la fecha de nacimiento), por lo tantos estos dos campos tienen que actualizarse coherentemente.
- **Proporcionan un sistema de seguridad** que garantiza el acceso a la información exclusivamente a aquellos usuarios que dispongan de autorización.
- **Proporcionan un diccionario de metadatos**, que contiene el esquema de la base de datos, es decir, cómo están estructurados los datos en tablas, registros y campos, las relaciones entre los datos, usuarios, permisos, etc.
- **Permiten el uso de transacciones**, es decir garantizan que todas las operaciones de la transacción se realicen correctamente, y en caso de alguna incidencia, deshacen los cambios ejecutados hasta el momento. Por ejemplo, si deseo hacer una transferencia bancaria, es importantísimo que, si se descuenta la cantidad a transferir de una cuenta, esa cantidad sea sumada en la cuenta destino, en caso de que el sistema se caiga en la mitad del procedimiento, es necesario que la base de datos vuelva a un estado consistente.
- Ofrecen herramienta que permiten **monitorizar el uso de la base de datos** y por lo tanto analizar sus posibles errores.
- **Permiten la concurrencia**, de tal manera que varios usuarios pueden trabajar sobre un mismo conjunto de datos garantizando el acceso correcto a los mismos, aunque otro usuario esté modificando esos mismos datos.
- Aseguran los tres niveles de abstracción del estándar ANSI, **garantizando por lo tanto la independencia de datos tanto física como lógica** explicada en el apartado anterior.
- **Ofrecen conectividad con el exterior**. Todos los SGBD incorporan herramientas estándar de conectividad. El protocolo ODBC está muy extendido como forma de comunicación entre bases de datos y aplicaciones externas.
- **Incorporan herramientas para la restauración de la información** en caso de que algún imprevisto que afecte o destruya la base de datos. El SGBD debe proporcionar mecanismos capaces de dejar la base de datos en un estado consistente lo más cercano posible al fallo.

3.2. Componentes de un SGBD

Los componentes más importantes de un SGDB son: el diccionario, los lenguajes de datos, los usuarios y el gestor de la base de datos.

Diccionario de datos:

El diccionario de datos o catálogo debe contener los metadatos, es decir, información que describa todos los datos de la base de datos. En él se describen entre otros el nombre, tipo y tamaño de cada uno de los datos, las posibles relaciones existentes entre ellos, las restricciones que deben de cumplir esos datos, qué usuarios que tienen acceso a qué datos, etc.

Este catálogo de datos debe contener la descripción de la información desde los tres niveles de abstracción, es decir, el interno, el conceptual y el externo.

Lenguaje de datos:

Una vez que tenemos creado el modelo de la aplicación, es decir, sé qué datos voy a guardar, cómo se van a relacionar esos datos, sus tipos, etc. necesito los lenguajes que me permitan no sólo definir cada elemento y restricción del modelo, sino también manipular sus datos y establecer los controles de acceso a los mismo.

Esto constituye los lenguajes de datos que se clasifican según su función es:

- **Lenguaje de definición de datos** (**DDL**, *Data Definition Language*): este lenguaje incluye las instrucciones que me van a permitir crear, eliminar y modificar la estructura de la base de datos (tablas, vistas, restricciones, etc.).
- **Lenguaje de manipulación de datos** (**DML**, *Data Manipulation Language*): este lenguaje incluye las instrucciones que me van a permitir manipular el contenido de la base de datos, es decir me va a permitir la inserción de datos nuevos en las tablas o vistas (estás tablas o vistas ya han sido previamente creadas con el lenguaje DDL), eliminación y actualización de esos datos o consulta de la información contenida en la base de datos.
- **Lenguaje de control de datos** (**DCL**, *Data Control Language*): este lenguaje incluye las instrucciones que me van a permitir controlar el acceso a la información y la seguridad de los datos. Por tanto, me permitirá establecer los permisos sobre los objetos (tablas, vistas, etc.) de las bases de datos, así como la creación y eliminación de cuentas de usuarios.

Usuarios de la base de datos:

En un SGBD intervienen distintos usuarios como son:

- **El administrador de la base de datos**: es la persona encargada de la seguridad y el control de los datos, definiendo para ello los esquemas físicos, lógicos y las vistas de usuario, asignando privilegios a los distintos usuarios, poniendo en marcha métodos de recuperación en caso de fallo, etc.
- **Los programadores de aplicaciones**: son las personas encargados de desarrollar las aplicaciones que van a permitir manejar de manera sencilla los datos de la base de datos.
- **Los usuarios finales**: son las personas que, sin la necesidad de tener conocimientos informáticos, podrán modificar y consultar los datos de la base de datos.

Gestor de la base de datos:

El gestor de la base de datos está formado por el conjunto de herramientas que me van a garantizar la seguridad, integridad, control de concurrencia, control de recuperación, importación/exportación de datos, etc.

3.3. Tipos de un SGBD

Existen numerosos tipos de SGBD que podemos clasificar según diversos criterios, destacando especialmente el SGBD basado en el modelo relacional, ya que gracias a su sencillez y flexibilidad se ha impuesto en casi todos los ámbitos del desarrollo.

Según el modelo de datos en el que se basa:

- **Jerárquico**: SGBD basado en el modelo de datos jerárquico visto en el apartado 2.4 de este tema.
- **En red**: SGBD basado en el modelo de datos en red explicado también anteriormente.
- **Relacional**. SGBD basado en el modelo de datos relacional donde se representan los datos y las relaciones entre estos a través de una colección de tablas. Ver apartado 2.4 para más información.

- **Orientado a objetos**: SGBD basado en el modelo de datos orientado a objeto. En este caso la base de datos está formada por objetos relacionados entre sí, siendo los objetos entidades con un estado o datos asociados, y un comportamiento o funcionalidad determinada.
- **Objeto-relacional**: extensión de los SGBD relacionales proporcionándole características de la programación orientada a objetos.

Según el número de usuarios que pueden acceder a la base de datos simultáneamente:

- **Monousuario**: solamente un usuario puede estar conectado.
- **Multiusuario**: varios usuarios pueden estar conectados simultáneamente.

Según el número de servidores en los que se encuentra almacenada la base de datos:

- **Centralizados**: en este tipo de SGBD todos sus componentes (datos, aplicaciones, etc.) se encuentran en un único servidor.
- **Distribuidos**: en este caso los componentes de los SGBD se almacenan en varios equipos servidores conectados por una red, pero que pueden encontrarse geográficamente separados.

Según el lenguaje soportado:

- **SQL**: los SGBD relacionales usan como principal lenguaje de programación el lenguaje SQL, hasta tal punto que se ha convertido un estándar en bases de datos relacionales. Por tanto, el lenguaje SQL nos va a permitir almacenar, manipular y recuperar datos almacenados en bases de datos relacionales.
- **NoSQL** (Not only SQL): este tipo de SGBD, de gran popularidad actualmente, se diferencia principalmente de los SGBD relacionales clásicos en que no utilizan el lenguaje de programación SQL. En los casos en los que es necesario acceder y analizar grandes cantidades de información almacenada en varios servidores en la red es especialmente útil estos SGBD NoSQL. Twitter y Facebook son dos ejemplos de aplicaciones que utilizan este tipo de bases de datos.

Según el tipo su tipo de licencia:

- **Comerciales**: SGBD distribuidos bajo licencias restrictivas, que me impiden la copia o modificación del mismo o el uso del programa para fines distintos de los establecidos.
  La mayoría de este tipo de programas suelen desarrollarse con fines lucrativos, es decir, para adquirir la licencia software que permita usar la aplicación, el usuario está obligado a pagar un determinado precio, aunque esto no siempre es así. Cada vez más fabricantes ofrecen versiones exprés, esto es versiones gratuitas que no libres.
  Algunos ejemplos de SGBD comerciales son SQL Server, Oracle, Microsoft Access.
- **Libres**: SGBD distribuidos bajo licencias no restrictivas, ya que no limitan los derechos que un usuario final puede hacer del software. Todo software libre debe permitir a todos los usuarios el acceso a su código fuente, por ello se denomina también software de código abierto.
  La mayoría de las veces este software es gratuito y se distribuye a través de Internet, aunque como siempre hay excepciones. La fuente de ingresos principalmente del software libre es la oferta de servicios adicionales de personalización, instalación, soporte técnico, etc.
  MySQL o PostgreeSQL son ejemplos de SGBD libres.

El uso del SGBD propietario no es mejor que el uso de SGBD libre, ni el SGBD libre mejor que el propietario, todo depende de las necesidades concretas y preferencias personales de cada usuario.

3.4. Bases de datos centralizadas y distribuidas

Como se ha explicado anteriormente existen dos tipos de SGBD dependiendo del número de servidores en los que se encuentra almacenada la base de datos, los SGBD centralizados y distribuidos.

- En un **SGBD centralizado** todos sus componentes (datos, aplicaciones, etc.) se encuentran en un único servidor. Los clientes que necesitan la información de esa base de datos se conectan a ese servidor a través de distintas interfaces.
- Sin embardo, en un **SGBD distribuido**, los componentes se almacenan en varios equipos servidores conectados por una red, pudiendo además estos servidores encontrarse ubicados en localidades geográficamente separadas.

«Una **base de datos distribuida** es una colección de datos que pertenece lógicamente al mismo sistema pero que se encuentra físicamente almacenada en distintas máquinas conectadas por una red».[1]

La fiabilidad, disponibilidad, distribución de la carga de trabajo, la facilidad de expansión y de compartir datos entre distintos usuarios son sólo algunas de las muchas ventajas que tienen las bases de datos distribuidas.

Todas estas ventajas, tienen asociada una mayor complejidad a la hora de diseñar, implementar y manipular los datos del SGBD, ya que la distribución de la base de datos en los distintos servidores tiene que ser un proceso completamente transparente para el usuario que accede a sus datos. Este usuario debe operar con la base de datos distribuida del mismo modo que lo haría en una centralizada.

Toda la información sobre la distribución final de la base de datos debe estar almacenada en el diccionario de datos.
Uno de los puntos clave a la hora de diseñar este tipo de bases de datos, es determinar qué datos se van a replicar y cuáles se van a fragmentar.

Fragmentación:

En una base de datos relacional (donde la información se almacena en tablas relacionadas entre sí), la fragmentación consiste en dividir una tabla en fragmentos más pequeños, de manera que cada fragmento se almacene en un lugar diferente. Se pueden realizar dos tipos de fragmentación: fragmentación horizontal y una fragmentación vertical.

En la **fragmentación horizontal** cada fragmento se corresponde con un subconjunto de filas de la tabla que cumplen una cierta condición.

[1] [1] Hueso Ibañez, L. (2012). Bases de datos. Editorial Ra-Ma. Pág. 36.

Ejemplo: la base de datos de un banco con sucursales por toda España puede estar formada por las tablas clientes y cuentas tal y como se muestra en las siguientes tablas:

Tabla cliente:

DNI	**Nombre**	**Apellido**	**Dirección**	**Ciudad**	**Provincia**
77228866	Pablo	Sánchez	C\ Alcalá, 28	Madrid	Madrid
66554477	Ana	Martín	C\ Pricesa, 30	Getafe	Madrid
44112233	Sonia	Alonso	P\ Mayor, 1	Santander	Santander
33665522	Pedro	Lorenzo	C\ Portugal, 23	Santander	Santander

Tabla cuentas:

Número cuenta	**DNI cliente**	**Saldo**	**Fecha apertura**
123	77228866	5000	12/03/2012
987	66554477	6000	15/08/1995
456	44112233	15800	14/04/2000
587	33665522	24000	15/11/2015

En vez de tener una base de datos instalada en un solo equipo servidor donde se encontrarían los datos de todos los clientes de España, puede resultar interesante almacenar los datos de cada cliente en un servidor que se encuentra en su provincia. De esta manera, el acceso a los datos sería mucho más rápido ya que la información de la que se dispone en cada tabla es mucho más pequeña, lo cual agiliza mucho las consultas y, además, está geográficamente más cercana al cliente.

Si por cualquier motivo desde una sucursal interesa acceder a los datos de un cliente de otra provincia, se podrá hacer sin problema, pero requerirá un poco más de tiempo de procesamiento.

De esta manera las tablas de la base de datos se almacenarían en servidores que se encuentran en distintas provincias tal y como se muestra a continuación:

Tablas clientes y cuentas de Madrid:

DNI	Nombre	Apellido	Dirección	Ciudad	Provincia
77228866	Pablo	Sánchez	C\ Alcalá, 28	Madrid	Madrid
66554477	Ana	Martín	C\ Pricesa, 30	Getafe	Madrid

Número cuenta	DNI cliente	Saldo	Fecha apertura
123	77228866	5000	12/03/2012
987	66554477	6000	15/08/1995

Tablas clientes y cuentas de Santander:

DNI	Nombre	Apellido	Dirección	Ciudad	Provincia
44112233	Sonia	Alonso	P\ Mayor, 1	Santander	Santander
33665522	Pedro	Lorenzo	C\ Portugal, 23	Santander	Santander

Número cuenta	DNI cliente	Saldo	Fecha apertura
456	44112233	15800	14/04/2000
587	33665522	24000	15/11/2015

En la **fragmentación vertical** cada fragmento se corresponde con un subconjunto de columnas (atributos) a los que las aplicaciones de usuario acceden normalmente de manera simultánea. Ojo, para poder recuperar la información completa en el momento que sea necesario, es necesario incluir un atributo común en todos los fragmentos (normalmente este atributo es una clave).

Ejemplo: la tabla cliente de la base de datos de un banco con sucursales por toda España que utilizábamos en el caso anterior, puede almacenar mucha más información de cada cliente, por ejemplo, la fecha de nacimiento, la nómina, sexo, estado civil, etc.

DNI	Nombre	Apellido	Dirección	Ciudad	Provincia	Fecha nac	Nómina	Sexo	Estado civil
77228866	Pablo	Sánchez	Alcalá, 28	Madrid	Madrid	11/08/1995	2000	H	Soltero
66554477	Ana	Martín	Pez, 30	Getafe	Madrid	12/06/1980	2500	M	Casado
44112233	Sonia	Alonso	Mayor, 1	Santander	Santander	07/02/1986	1300	M	Casado
33665522	Pedro	Lorenzo	Gloria, 23	Santander	Santander	09/12/1975	1600	H	Divorciado

La mayoría de las aplicaciones de usuario utilizarán solamente el DNI, nombre apellido, dirección, ciudad y provincia. El resto de los datos serán accedidos por otras aplicaciones de uso mucho menos frecuente. En este caso se podría realizar una fragmentación vertical de esta tabla, en la que tendría en un servidor más potente los datos de los atributos de mayor acceso (el DNI, nombre apellido, dirección, ciudad y provincia) y en otro servidor con menos capacidad de

procesamiento, pero a lo mejor más capacidad de almacenamiento podría almacenar el resto de los datos.

En este caso las tablas fragmentadas verticalmente quedarían de la siguiente manera (atención, hay que añadir un atributo común que me permita reestructurar la tabla completa).

DNI	Nombre	Apellido	Dirección	Ciudad	Provincia
77228866	Pablo	Sánchez	Alcalá, 28	Madrid	Madrid
66554477	Ana	Martín	Pez, 30	Getafe	Madrid
44112233	Sonia	Alonso	Mayor, 1	Santander	Santander
33665522	Pedro	Lorenzo	Gloria, 23	Santander	Santander

DNI	Fecha nac	Nómina	Sexo	Estado civil
77228866	11/08/1995	2000	H	Soltero
66554477	12/06/1980	2500	M	Casado
44112233	07/02/1986	1300	M	Casado
33665522	09/12/1975	1600	H	Divorciado

Existe también la posibilidad de una fragmentación en el que se incluyan los dos casos anteriores, fragmentación horizontal y fragmentación vertical. A esto se llama **fragmentación mixta**.

Replicación:

La replicación como su propio nombre indica consiste en almacenar en equipos servidores distintos varias copias idénticas de una o varias tablas.

Las ventajas fundamentales de la replicación son el aumento de la disponibilidad, ya que en caso de caída de uno de los servidores el sistema seguía funcionando y el aumento del paralelismo, lo que agilizará mucho las consultas de datos.

Si en nuestra base de datos ser realizan muchas actualizaciones de datos, las ventajas de la replicación se ven considerablemente disminuidas. El sistema debe asegurar la integridad y consistencia de datos en todas las réplicas, por lo que una actualización en un dato debe propagarse a todas las copias de ese dato para asegurarnos que los clientes siempre tienen la información correcta. Esto puede llevarnos mucho tiempo, por lo que hay que barajar siempre si es necesario replicar y en qué medida, o no.

04. Bibliografía

Hueso Ibañez, L. (2012). Bases de datos. Editorial Ra-Ma.

López Montalbán, I. y de Castro Vázquez, M. (2014). Gestión de bases de datos. Editorial Garceta.

Date, C. J. (2001). Introducción a los sistemas de bases de datos. Editorial Pearson Prentice Hall.

De Miguel, A.; Piattini, M.; Marcos, E. (1999). Diseño de bases de datos relacionales. Editorial Ra-Ma.